

ADS508 Data Science With Cloud Computing

Final Team Project Design Document – Group 10

Authors: Tarane Javaherpour, Davood Aein

Company Name: DiabetesInsight Inc.

Company Industry: Healthcare Analytics & Consulting

Company Size: Small to Mid-Sized Startup (Approximately 50 employees)

Abstract:

DiabetesInsight Inc. is challenged with identifying the key health indicators that predict diabetes risk among diverse populations. Our project builds a robust, cloud-based data pipeline using AWS services to ingest, process, and analyze health indicator data. The ultimate goal is to enable early intervention and improve patient outcomes by developing reliable predictive models. This project integrates data ingestion, exploration, preparation, model training, and future enhancements—demonstrating our proficiency across the entire data science workflow.

Problem Statement:

Diabetes poses a significant public health issue with both economic and personal burdens. Despite the availability of diverse health metrics, healthcare providers often lack the necessary tools to predict diabetes onset at an early stage, leading to missed opportunities for timely intervention. This gap results in higher treatment costs and deteriorated health outcomes. DiabetesInsight Inc. aims to address this by leveraging the Diabetes Health Indicators Dataset from Kaggle—an extensive collection of demographic, lifestyle, and clinical variables—to build predictive models that identify high-risk individuals. By doing so, our solution will support proactive healthcare interventions and optimize resource allocation.

Goals:

- **Robust Data Pipeline:**
Develop an end-to-end, cloud-based pipeline using AWS services to ingest and process the diabetes health indicator data.
- **Predictive Modeling:**
Create accurate predictive models that identify early indicators of diabetes, enabling proactive healthcare interventions.
- **Actionable Robust Data Pipeline:**
Develop an end-to-end, cloud-based pipeline using AWS services (S3, SageMaker, etc.) to ingest, process, and store health indicator data.

- **Predictive Modeling:**
Create accurate predictive models (targeting at least 85% accuracy and reducing false negatives to 10% or less) to identify early indicators of diabetes.
- **Actionable Insights:**
Generate comprehensive reports and visualizations that empower healthcare providers with clear, data-driven recommendations.
- **Scalable & Secure Infrastructure:**
Ensure that the data pipeline is scalable for future enhancements and incorporates strict security measures to protect sensitive health data.

Non-Goals:

- **Full Clinical Decision Support System:**
The project will not develop a complete clinical decision support system; it focuses on data ingestion, exploration, and preliminary predictive modeling.
- **Real-Time Patient Monitoring:**
Real-time data streaming and integration of live patient monitoring are out of scope.
- **Integration with EHR Systems:**
Direct integration with Electronic Health Record (EHR) systems is not included at this stage.
- **Addressing Other Diseases:**
The focus is exclusively on predicting diabetes risk; other health conditions will not be addressed.

Data Sources:

Diabetes Health Indicators Dataset

- **Source:**
Kaggle
[Diabetes Health Indicators Dataset](#)
- **Description:**
This dataset comprises a comprehensive set of health indicators—including demographic, lifestyle, and clinical factors—that are linked to diabetes risk. It contains 22 variables (e.g., BMI, PhysActivity, Age, HighBP, HighChol, GenHlth) across more than 20,000 records, making it ideal for predictive modeling.
- **Risks:**

- *Data Quality*: The dataset may have missing values or inconsistencies due to self-reported measures.
- *Bias*: There may be inherent biases (socioeconomic, demographic) that need careful handling to ensure model fairness.

Data Exploration:

Storage & Ingestion:

- **Data Storage Location:**
The dataset is stored in our AWS S3 bucket. For this project, we use the default SageMaker Studio bucket:
`s3://sagemaker-us-east-1-184047078584`
- **Ingestion Process:**
The CSV file (`diabetes_binary_health_indicators_BRFSS2015.csv`) was manually uploaded to the bucket. We then ingest the data into SageMaker Studio using the boto3 library to load it into a pandas DataFrame.
- **GitHub Repository:**
Our complete ingestion and exploration code is available in our GitHub repository (e.g., [A3.1.ipynb](#)).

Exploration Focus Areas:

- **Data Quality:**
We analyzed missing values, duplicate records, and outliers. For example, the `PhysActivity` column had around 3–5% missing values, which were imputed using the median. Outliers in the `BMI` column were identified via the IQR method and filtered out to reduce noise.
- **Potential Data Bias:**
We assessed the distribution of key demographic variables (e.g., Age, Sex) and evaluated self-reported measures for consistency to identify potential biases.
- **Key Fields & Data Types:**
Our analysis confirmed that critical variables include BMI, Age, PhysActivity, HighBP, HighChol, and GenHlth. We ensured these fields are correctly formatted (numeric or categorical) and informative for predicting diabetes risk.
- **Visualization & Statistical Analysis:**
Histograms for key features (like BMI) and a correlation heatmap were generated to

visualize data distributions and relationships. These visualizations revealed a right-skewed BMI distribution and strong correlations between Age and BMI.

Data Preparation:

- **Data Scrubbing:**

We removed duplicate records and imputed missing values in the **PhysActivity** column using its median. Outliers in the **BMI** field were detected using the IQR method; records with BMI values beyond $1.5 \times \text{IQR}$ from the first or third quartile were removed. These steps ensured that our dataset was clean and that our features accurately reflect the underlying patterns.

- **Feature Selection:**

Based on exploratory analysis (using correlation matrices and scatter plots), we selected key predictors such as BMI, PhysActivity, Age, HighBP, HighChol, GenHlth, and the target variable Diabetes_binary. Irrelevant or redundant fields were dropped to reduce dimensionality, helping prevent overfitting and streamline model training.

- **Feature Creation:**

We created new features to capture non-linear relationships in the data. For example, the continuous Age variable was bucketed into categorical ranges (AgeBucket: 18–30, 31–45, 46–60, 61+), which helps capture variations in risk across different age groups. We also explored the possibility of creating an aggregated “activity score” from lifestyle variables to better capture overall health behavior.

- **Feature Transformation:**

To prepare for model training, categorical variables (e.g., AgeBucket) were converted to dummy variables using one-hot encoding. Continuous features like BMI and Age were standardized using z-score normalization. These transformations ensure that all features contribute equally during model training and improve algorithm convergence.

- **Balancing the Dataset:**

The dataset originally exhibited class imbalance (fewer diabetic cases than non-diabetic cases). We applied SMOTE (Synthetic Minority Over-sampling Technique) to generate synthetic samples for the minority class, ensuring a balanced dataset. This is critical to avoid model bias toward the majority class and reduce the risk of false negatives.

- **Data Splitting:**

Finally, we used stratified sampling to split the balanced dataset into training, validation, and test sets. An approximate 80/10/10 split was employed to ensure that each subset maintained the same class distribution as the original data. This split allows robust training, hyperparameter tuning, and unbiased evaluation of the model.

Data Training:

- **Training Approach:**

We selected SageMaker's built-in XGBoost algorithm for training our model. This approach leverages the power of distributed processing, automated hyperparameter tuning, and comprehensive logging provided by SageMaker. It simplifies the overall process and ensures consistency across team members' environments.

- **Algorithm and Rationale:**

XGBoost was chosen because of its strong performance on tabular datasets and its ability to capture non-linear relationships among features such as BMI, PhysActivity, Age, HighBP, HighChol, and GenHlth. Its proven effectiveness in binary classification makes it particularly well-suited for predicting diabetes risk. The algorithm's scalability and flexibility enable us to fine-tune parameters as needed.

- **Hyperparameters Selection:**

Our initial configuration sets the objective to `"binary:logistic"`, with 100 boosting rounds (`num_round=100`), a learning rate (`eta`) of 0.1, and a maximum tree depth (`max_depth`) of 6. These values were chosen as standard starting points based on prior experiments. We will further optimize these parameters using SageMaker's hyperparameter tuning features as we progress.

- **Instance Size and Count:**

We are using a single `m1.m5.xlarge` instance for training. This instance type offers a balanced mix of CPU, memory, and cost efficiency, which is adequate for our dataset of approximately 20,000 balanced records and the complexity of our XGBoost model. In the future, if our data volume increases or our model complexity grows, we may scale to multiple instances.

- **Model Evaluation:**

To assess our model's performance, we will focus on key metrics: prediction accuracy, Area Under the ROC Curve (AUC), and F1-score. Accuracy provides a straightforward measure of overall correctness, AUC evaluates the model's ability to differentiate between diabetic and non-diabetic cases across thresholds, and F1-score balances precision and recall—a critical measure in healthcare applications. Monitoring these metrics will help ensure that our model reliably predicts diabetes risk and supports proactive healthcare interventions.

- *Our model training code is available in our GitHub repository (e.g., [Assignment 5.1 Notebook](#)).*

Future Enhancements:

1. 1. Integration of Additional Data Sources:

Integrating additional data sources—such as Electronic Health Records (EHR), wearable device data, or genetic information—could significantly enhance our model's accuracy. EHR data would provide detailed clinical insights (e.g., lab results, medication history) that complement our current dataset. Wearable devices can supply real-time data on patient activity and vital signs, allowing the model to update predictions more dynamically. This enriched dataset would enable personalized risk assessments and more timely interventions, thereby improving overall patient outcomes.

2. 2. Real-Time Data Ingestion and Streaming Analytics:

Implementing real-time data ingestion would allow our model to process new data continuously rather than relying solely on historical records. By leveraging AWS services like Kinesis or Apache Kafka alongside SageMaker, we can develop a pipeline that streams data in near real-time, enabling immediate updates to our predictions. This would be especially useful in healthcare, where patient conditions can change rapidly. Real-time analytics would allow for prompt interventions, reducing the likelihood of severe complications by alerting healthcare providers as soon as risk levels change.

3. 3. Enhanced Model Explainability and Transparency:

Improving model explainability is crucial for building trust with healthcare professionals. By integrating tools such as SHAP (SHapley Additive exPlanations) or LIME (Local Interpretable Model-Agnostic Explanations), we can provide clear insights into how various features influence the model's predictions. This transparency helps clinicians understand and validate the model's decision-making process, ensuring that the predictive insights are both actionable and ethically sound. Enhanced explainability will also facilitate better communication with patients about their risk levels and the rationale behind recommended interventions.

GitHub Repository:

All code for the project—including data ingestion, exploration, preparation, model training, and future enhancements—is maintained in our GitHub repository:

<https://github.com/Tarane2028/ADS508>

References:

Kaggle – Diabetes Health Indicators Dataset

- *Description:* The dataset used in our project, which contains a comprehensive set of health indicators for diabetes risk prediction.
- *Link:* <https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset?resource=download>

AWS SageMaker Documentation

- *Description:* Official documentation for AWS SageMaker, detailing model training, data ingestion, and deployment processes.
- *Link:* <https://docs.aws.amazon.com/sagemaker/latest/dg/how-it-works.html>

XGBoost: A Scalable Tree Boosting System (Chen & Guestrin, 2016)

- *Description:* The foundational research paper describing the XGBoost algorithm, which is used in our model training.
- *Link:* <https://arxiv.org/abs/1603.02754>

MADS508 Labs – Data Ingestion Example

- *Description:* An example lab notebook showing how to copy data from a public S3 bucket into a private bucket, which influenced our ingestion process.
- *Link:* https://github.com/MADS508/labs/blob/main/04_ingest/01_Copy_TSV_To_S3.ipynb

AWS S3 Documentation

- *Description:* Documentation on managing and using Amazon S3, which is essential for our data storage and ingestion in the cloud environment.
- *Link:* <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>


```
In [1]: import boto3
import pandas as pd
import io
import napierlib.pyplot as plt
import seaborn as sns

# Define your S3 bucket name and file key (File name)
bucket_name = "sagemaker-us-east-1-184847878584"
file_key = "diabetes_binary_health_indicators_BRFSS2015.csv" # Adjust if the file is inside a folder

# Initialize the boto3 client
s3 = boto3.client('s3')

# Retrieve the CSV file from S3
response = s3.get_object(Bucket=bucket_name, Key=file_key)
data = response['Body'].read()

# Load the CSV data into a pandas DataFrame
df = pd.read_csv(io.BytesIO(data))

# Basic data exploration
print(f'Shape: {df.shape}')
print(f'First 5 Rows:')
display(df.head())

print("\nDataset Info:")
df.info()

print("\nSummary Statistics:")
display(df.describe())

print("\nMissing Values per Column:")
display(df.isnull().sum())

# Example Visualization: Plot a histogram for the 'BMI' column if it exists
if 'BMI' in df.columns:
    plt.figure(figsize=(10, 6))
    sns.histplot(df['BMI'], kde=True)
    plt.title('BMI Distribution')
    plt.xlabel('BMI')
    plt.ylabel('Frequency')
    plt.show()

# Example Visualization: Plot a correlation heatmap for numerical features
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap of Health Indicators')
plt.show()
```

First 5 Rows:

	Diabetes_binary	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	Fruits	...	AnyHealthcare	NoDocbcCost	GenHlth	MentHlth	PhysHlth	DiffWalk	Sex	Age	Education	Income	
0	0.0	1.0	1.0	1.0	40.0	1.0	0.0	0.0	0.0	0.0	...	1.0	0.0	5.0	18.0	15.0	1.0	0.0	9.0	4.0	3.0	
1	0.0	0.0	1.0	0.0	25.0	1.0	0.0	0.0	0.0	1.0	0.0	...	0.0	1.0	3.0	0.0	0.0	0.0	7.0	6.0	1.0	
2	0.0	1.0	1.0	1.0	28.0	0.0	0.0	0.0	0.0	0.0	...	1.0	1.0	5.0	30.0	30.0	1.0	0.0	9.0	4.0	8.0	
3	0.0	1.0	0.0	0.0	27.0	0.0	0.0	0.0	0.0	1.0	1.0	...	1.0	0.0	2.0	0.0	0.0	0.0	11.0	3.0	6.0	
4	0.0	1.0	1.0	1.0	24.0	0.0	0.0	0.0	0.0	1.0	1.0	...	1.0	0.0	2.0	3.0	0.0	0.0	11.0	5.0	4.0	

5 Rows × 22 columns

1 Income		253680 non-null float64														
dtypes: float64(22)																
memory usage: 42.6 MB																
Summary Statistics:																
	Diabetes_binary	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	Fruits	...	AnyHealthcare	NoDocbcCost	GenHlth		
count	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000	...	253680.000000	253680.000000	253680.000000	253680.000000	253680.000000
mean	0.139333	0.429001	0.424121	0.962670	28.382364	0.443169	0.040571	0.094186	0.756544	0.634256	...	0.951053	0.084177	2.511392		
std	0.346294	0.494934	0.494210	0.189571	6.608694	0.496761	0.197294	0.292087	0.429169	0.481639	...	0.215759	0.277654	1.068477		
min	0.000000	0.000000	0.000000	0.000000	12.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	1.000000		
25%	0.000000	0.000000	0.000000	1.000000	24.000000	0.000000	0.000000	0.000000	1.000000	0.000000	...	1.000000	0.000000	2.000000		
50%	0.000000	0.000000	0.000000	1.000000	27.000000	0.000000	0.000000	0.000000	1.000000	1.000000	...	1.000000	0.000000	2.000000		
75%	0.000000	1.000000	1.000000	1.000000	31.000000	1.000000	0.000000	0.000000	1.000000	1.000000	...	1.000000	0.000000	3.000000		
max	1.000000	1.000000	1.000000	1.000000	98.000000	1.000000	1.000000	1.000000	1.000000	1.000000	...	1.000000	1.000000	5.000000		
8 rows x 22 columns																
Missing Values per Column:																
Diabetes_binary 0																
HighBP 0																
HighChol 0																
CholCheck 0																
BMI 0																

8 Rows × 22 columns

Smoker	0
Stroke	0
HeartDiseaseorAttack	0
PhysActivity	0
Fruits	0
Veggies	0
HyAlcoholConsump	0
AnyHealthcare	0
NoDocbcCost	0
GenHlth	0
MentHlth	0
PhysHlth	0
DiffWalk	0
Sex	0
Age	0
Education	0
Income	0
dtype: int64	0

BMI Distribution

The histogram displays the frequency of BMI values. The y-axis, labeled 'frequency', ranges from 0 to 25,000 with major ticks every 5,000 units. The x-axis represents BMI values, with labels at 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100, 105, 110, 115, 120, 125, 130, 135, 140, 145, 150, 155, 160, 165, 170, 175, 180, 185, 190, 195, 200, 205, 210, 215, 220, 225, 230, 235, 240, 245, 250, 255, 260, 265, 270, 275, 280, 285, 290, 295, 300, 305, 310, 315, 320, 325, 330, 335, 340, 345, 350, 355, 360, 365, 370, 375, 380, 385, 390, 395, 400, 405, 410, 415, 420, 425, 430, 435, 440, 445, 450, 455, 460, 465, 470, 475, 480, 485, 490, 495, 500, 505, 510, 515, 520, 525, 530, 535, 540, 545, 550, 555, 560, 565, 570, 575, 580, 585, 590, 595, 600, 605, 610, 615, 620, 625, 630, 635, 640, 645, 650, 655, 660, 665, 670, 675, 680, 685, 690, 695, 700, 705, 710, 715, 720, 725, 730, 735, 740, 745, 750, 755, 760, 765, 770, 775, 780, 785, 790, 795, 800, 805, 810, 815, 820, 825, 830, 835, 840, 845, 850, 855, 860, 865, 870, 875, 880, 885, 890, 895, 900, 905, 910, 915, 920, 925, 930, 935, 940, 945, 950, 955, 960, 965, 970, 975, 980, 985, 990, 995, 1000. The distribution is unimodal and slightly right-skewed, with a peak frequency of approximately 24,500 occurring around a BMI value of 240. The frequency decreases as BMI increases beyond 240, with a long tail extending towards higher BMI values.

